

Objectif

Savoir implémenter un service REST. Savoir traiter les différentes requêtes et faire le lien avec la base de données.

1 Le restaurant de pizzas

On souhaite ici développer une application REST pour un restaurant de pizzas. Cette application comprendra à terme, une ressource `Ingredients` qui correspond aux ingrédients possibles dans une pizza, une ressource `Pizzas` qui correspond au catalogue de pizzas avec pour chacune, un ensemble d'ingrédients. Enfin, nous aurons une ressource `Commandes` qui correspond à une commande effectuée avec les renseignements sur le client et les pizzas commandées.

1.1 L'ossature de l'application WEB

Afin de réaliser cette application nous allons bien évidemment créer un nouveau contexte.

1. Créez dans `webapps` un nouveau contexte nommé `pizzeria`.
2. Recopiez d'un autre contexte comme `vide` le répertoire `META-INF` et son contenu
3. Dans `WEB-INF` créez un répertoire `classes` et un répertoire `src`. Il contiendra tous les sources utilisés
4. Dans `src` créez les répertoires `dto`, `dao`, `controleurs` qui contiendront les objets correspondants.

Vous devriez obtenir l'arborescence suivante

```
webapps
|- vide
|- ....
|- pizzeria
    |- META-INF
    |   | context.xml
    |- WEB-INF
        |- classes
        | src
            |- dto
            |- dao
            |- controleurs
```

5. Assurez vous d'avoir accès aux dépendances `servlet`, `jackson-annotations`, `jackson-core`, `jackson-databind`

1.2 Une première approche avec la ressource Ingrédients

Nous commençons ce projet avec la ressource `Ingredient`.

1. Par soucis de simplification, dans un premier temps, on se contentera d'un DAO "trivial" qui, au lieu de se connecter à la base de données, gère une liste d'ingrédients (ou éventuellement une Map) créés et ajoutés "en dur" dans le constructeur de ce DAO (comme dans le `tp321`).
 - Un ingrédient est caractérisé par un identifiant `id` de type `int` un `nom` de type `String` et un `prix` de type `int`. Créez un DTO/POJO `Ingredient.java` avec ces trois attributs et leurs accesseurs.
 - Créez un `IngredientDAOList.java`, qui gère une liste d'ingrédients initialement ajoutés dans le constructeur de ce DAO avec quelques ingrédients (`poivrons`, `chorizo`, `lardons`)...
On écrira trois méthodes dans ce modèle : `findAll` qui renvoie la liste de tous les ingrédients, `findById` qui renvoie l'ingrédient correspondant à la clé fournie en paramètre, et `save` qui permet d'ajouter un nouvel ingrédient à la liste.
2. Créez une servlet contrôleur `IngredientRestAPI.java`. Dotez cette classe d'un endpoint `/ingrédients` qui renvoie la collection d'ingrédients en JSON. Un ingrédient sera classiquement représenté en JSON par la chaîne

```
{
  "id" : 123,
  "nom" : "mozzarella"
  "prix" : 2
}
```

C'est Jackson et sa méthode `writeValueAsString` qui s'occupera de la conversion Objet - JSON

3. Une fois cette méthode réalisée, vérifiez que
`curl -i localhost:8080/pizzeria/ingredients`
vous renvoie bien la liste de vos ingrédients en JSON
4. On souhaite que notre API puisse maintenant répondre aux requêtes suivantes :

URI	HTTP	Requête	Réponse
<code>/ingrédients</code>	GET		liste des ingrédients
<code>/ingrédients/{id}</code>	GET		l'ingrédient indiqué ou 404
<code>/ingrédients</code>	POST	ingrédient JSON	l'ingrédient créé ou 409 si il existe déjà (même id)

Complétez votre contrôleur avec ces autres endpoints

5. Vérifiez avec `curl` ou `httpie` que vous êtes capables d'ajouter `ananas`, `fromage`, `tomates`, `olives`, `oeufs` et que vous savez les récupérer.

```
http GET http://localhost:8080/pizzeria/ingredients
curl -i -X GET http://localhost:8080/pizzeria/ingredients
```

```
http GET http://localhost:8080/pizzeria/ingredients/2
curl -i -X GET http://localhost:8080/pizzeria/ingredients/2
```

```
http GET http://localhost:8080/pizzeria/ingredients/22
curl -i -X GET http://localhost:8080/pizzeria/ingredients/22
```

```
curl -i -X POST http://localhost:8080/pizzeria/ingredients -d '{"id":3 ,
"name":"chorizo" , "prix":3}'
http POST http://localhost:8080/pizzeria/ingredients id=3 name="chorizo"
```

```
http GET http://localhost:8080/pizzeria/ingredients
curl -i -X GET http://localhost:8080/pizzeria/ingredients
```

1.3 Le lien avec la base de données

Pour que l'application puisse prendre de l'ampleur, on souhaite remplacer la `List` par une table de base de données

1. Créez une table `ingredients(id,nom,prix)` dans la base de données, avec quelques ingrédients dedans (pomme de terre, poivrons, chorizo, lardons, aubergines, champignons)

```
CREATE TABLE ingredients(id int, nom text,prix int);
INSERT INTO ingredients VALUES (1,'pomme de terre',3);
INSERT INTO ingredients VALUES (2,'poivrons',2);
INSERT INTO ingredients VALUES (3,'chorizo',4);
INSERT INTO ingredients VALUES (4,'lardons',4);
INSERT INTO ingredients VALUES (5,'aubergines',2);
INSERT INTO ingredients VALUES (6,'champignons',2);
```

2. Créer une interface `DAOIngredient.java` et faites en sorte d'avoir deux implémentations : `IngredientDAO` (faite précédemment) et `IngredientDAODatabase` (à faire).
3. Vérifiez (à la fois via `curl` et en regardant dans la base de données) que vous êtes capables d'ajouter `ananas`, `fromage`, `tomates`, `olives`, `oeufs` et que vous savez les récupérer (en faisant un appel à `findAll` par exemple).